

# bcrypt: como funciona o algoritmo de hash de senhas

Documento de estudo para o Seminario 01 de Seguranca (Algoritmos de Criptografia), Tema 09: bcrypt. Autores do trabalho: Isabela Sousa e Paulo Victor Almeida. O objetivo deste texto e explicar, em linguagem simples e em ordem logica, o que e o bcrypt, por que ele existe, como o algoritmo funciona passo a passo e quais sao seus pontos fortes e limitacoes. Tambem descreve a implementacao feita pela dupla em Python puro, sem nenhum import.

## 1. O problema: como guardar senhas

Qualquer sistema com login precisa verificar senhas. A pior solucao e guardar a senha em texto puro: se o banco de dados vazar, todas as senhas ficam expostas. A solucao correta e guardar apenas um hash da senha. Um hash e uma funcao de mao unica: dada a senha, e facil calcular o hash; dado o hash, nao existe forma pratica de recuperar a senha. No login, o sistema calcula o hash da senha digitada e compara com o hash guardado.

O problema aparece quando o banco vaza. O atacante nao consegue desfazer o hash, mas pode chutar senhas: calcula o hash de milhoes de palavras comuns e compara com os hashes roubados. Esse ataque se chama forca bruta ou ataque de dicionario. Funcoes de hash de uso geral, como MD5 e SHA-256, foram projetadas para serem rapidas, e por isso um atacante com placas de video consegue testar bilhoes de palpites por segundo. Para senhas, e preciso o oposto: uma funcao lenta de proposito.

## 2. O que e o bcrypt

O bcrypt e uma funcao de hash feita especificamente para senhas. Foi criado por Niels Provos e David Mazieres, apresentado em 1999 na conferencia USENIX no artigo 'A Future-Adaptable Password Scheme', e foi desenvolvido para o sistema operacional OpenBSD. Ele e baseado na cifra de bloco Blowfish, criada por Bruce Schneier em 1993. O bcrypt reaproveita a parte mais cara do Blowfish, a preparacao da chave, e a transforma em um mecanismo de lentidao ajustavel.

O bcrypt tem tres ideias centrais. A primeira e o hash de mao unica: nao existe operacao de descriptografar. A segunda e o sal: um valor aleatorio de 16 bytes (128 bits) gerado para cada senha e guardado dentro do proprio hash. Gracias ao sal, a mesma senha gera hashes diferentes em contas diferentes, e as tabelas pre-calculadas chamadas rainbow tables deixam de funcionar. O sal nao e segredo. A terceira ideia e o custo: um numero N que faz o algoritmo repetir o trabalho  $2^N$  vezes. Cada aumento de 1 no custo dobra o tempo de calculo. Assim, conforme os computadores ficam mais rapidos, basta aumentar o custo. A recomendacao atual e usar custo 12 ou mais, com minimo de 10.

## 3. Anatomia do hash de 60 caracteres

Um hash bcrypt tem sempre 60 caracteres e esta no formato: cifrao, versao, cifrao, custo com dois digitos, cifrao, 22 caracteres de sal e 31 caracteres de hash. Exemplo: \$2b\$05\$RJatSMavAUXuKMySN8WKfeOeoSNcdW5.UBnEOm9EnZpnpV64Fe0eq. Aqui '2b' e a

versao do algoritmo, '05' e o custo (2 elevado a 5, ou seja, 32 repeticoes), 'RJatSMavAUXuKMySN8WKfe' e o sal de 16 bytes codificado em 22 caracteres e o restante e o hash de 23 bytes codificado em 31 caracteres. Como versao, custo e sal ficam dentro do proprio hash, o sistema de login consegue refazer o calculo usando apenas o hash armazenado e a senha digitada.

A codificacao usa um base64 proprio do bcrypt, com o alfabeto de ponto, barra, letras maiusculas, letras minusculas e digitos, e sem o sinal de igual de preenchimento. A cada 3 bytes sao gerados 4 caracteres. Por isso 16 bytes de sal viram 22 caracteres e 23 bytes de hash viram 31 caracteres.

## 4. A base: a cifra Blowfish

O Blowfish e uma cifra de bloco: cifra dados em blocos de 64 bits, vistos como duas metades de 32 bits, a esquerda e a direita. Usa uma estrutura chamada rede de Feistel, com 16 rodadas. Em cada rodada, a metade esquerda recebe um XOR com uma subchave, o resultado passa por uma funcao F e e combinado por XOR com a metade direita, e depois as duas metades trocam de lugar. No final ha um passo de branqueamento com duas subchaves extras.

O estado interno do Blowfish tem duas partes: o vetor P, com 18 subchaves de 32 bits, e quatro S-boxes, que sao tabelas de 256 numeros de 32 bits cada, totalizando cerca de 4 KB. A funcao F divide uma palavra de 32 bits em quatro bytes a, b, c e d, consulta uma S-box para cada byte e combina os resultados:  $F(x) = ((S0[a] + S1[b]) \text{ XOR } S2[c]) + S3[d]$ , com somas modulo 2 elevado a 32.

Os valores iniciais do vetor P e das S-boxes sao os digitos hexadecimais da parte fracionaria do numero pi. Por exemplo, P[0] vale 243F6A88. Usar pi mostra que nao ha nenhuma porta dos fundos escondida nos numeros, pois qualquer pessoa pode conferir. Na implementacao da dupla, esses digitos nao sao colados de uma tabela: sao calculados com a formula de Machin,  $\pi = 16 \arctan(1/5) - 4 \arctan(1/239)$ , usando apenas aritmetica de inteiros grandes.

O ponto que interessa ao bcrypt: preparar uma chave no Blowfish e caro. A preparacao mistura a chave no vetor P e depois cifra blocos repetidamente para reescrever todo o vetor P e todas as S-boxes. Sao 9 cifragens para o vetor P e 512 para as quatro S-boxes, num total de 521 cifragens por preparacao.

## 5. O algoritmo do bcrypt passo a passo

O bcrypt usa uma variante chamada Eksblowfish, de 'expensive key schedule Blowfish', ou seja, Blowfish com preparacao de chave cara. As entradas sao a senha (ate 72 bytes), um sal de 16 bytes e o custo. Os passos sao seis.

Passo 1, preparar a senha: a senha e convertida em bytes UTF-8, recebe um byte nulo no final e e cortada em 72 bytes. O numero 72 vem do tamanho do vetor P: 18 palavras de 4 bytes. Tudo o que passar disso e ignorado.

Passo 2, estado inicial: o vetor P e as quatro S-boxes sao carregados com os digitos de pi.

Passo 3, misturar sal e senha: executa-se a operacao ExpandKey com o sal e a senha. Nela, cada palavra do vetor P recebe um XOR com palavras da chave (a chave e repetida em ciclo ate

cobrir as 18 palavras). Depois cifra-se um bloco zerado; o resultado substitui P[0] e P[1]; cifra-se esse resultado para obter P[2] e P[3], e assim por diante, até reescrever o vetor P e as quatro S-boxes. O sal, dividido em quatro palavras de 32 bits, entra por XOR no bloco antes de cada cifração. São 521 cifrações.

Passo 4, o coração do algoritmo e onde está a lentidão: repete-se 2 elevado ao custo vezes duas operações, primeiro ExpandKey apenas com a senha e depois ExpandKey apenas com o sal. Como cada cifração depende do estado já alterado pela anterior, não é possível paralelizar nem pular etapas.

Passo 5, gerar a saída: o texto fixo de 24 bytes 'OrpheanBeholderScryDoubt', que forma 3 blocos de 64 bits, e cifrado 64 vezes com o estado final. O resultado tem 24 bytes, dos quais se descarta o último, ficando 23 bytes.

Passo 6, montar o hash: concatena-se o prefixo com a versão, o custo, o sal em base64 e os 23 bytes em base64, gerando a string de 60 caracteres que é guardada no banco.

Contagem de trabalho: o número total de cifrações Blowfish é  $(1 + 2 \text{ vezes } 2 \text{ elevado ao custo}) \text{ vezes } 521$ , mais 192 das cifrações finais (64 repetições de 3 blocos). Com custo 4 são 17.385 cifrações; com custo 5, 34.057; com custo 12, cerca de 4,3 milhões.

## 6. Exemplo prático

No exemplo do trabalho, a senha 'senhaeu' é processada com custo 5. A chave tem 8 bytes (a senha mais o byte nulo). O primeiro valor do vetor P começa em 243F6A88 (dígitos de  $\pi$ ), passa a 74F1E7CD depois de misturar sal e senha, e chega a 03CCDF0A depois das 32 repetições. Ao final, 64 cifrações do texto fixo produzem os bytes que, em base64, formam o hash. No total são 34.057 cifrações Blowfish. Trocar uma única letra da senha muda o hash por completo.

No login, o sistema lê o custo e o sal de dentro do hash guardado, refaz o cálculo com a senha digitada e compara o resultado com o hash esperado. Nunca há descryptografia: o sistema apenas recalcula e compara.

## 7. Por que o bcrypt é seguro

O bcrypt não impede o atacante de chutar senhas; ele torna cada palpite caro. O sal único por senha obriga o atacante a atacar cada usuário separadamente e inutiliza rainbow tables. O custo acompanha a evolução do hardware. As etapas em sequência impedem paralelização dentro de um único hash. Além disso, a verificação deve comparar o hash calculado com o esperado em tempo constante: em vez de parar no primeiro byte diferente, percorre todos os bytes, para que o tempo de resposta não revele quantos bytes o atacante acertou, o que seria um ataque de temporização (timing attack).

Importante: o bcrypt não salva uma senha fraca. Senhas como '123456' ou 'password' caem em qualquer dicionário. O bcrypt apenas encarece cada tentativa.

## 8. Limitações

A primeira limitação é o limite de 72 bytes: duas senhas que sejam iguais nos primeiros 72 bytes geram o mesmo hash, pois o restante é ignorado. Uma solução comum é aplicar um pre-hash na

senha antes do bcrypt. A segunda é que o byte nulo não pode aparecer na senha, pois ele é usado como terminador. A terceira é o uso de pouca memória, cerca de 4 KB fixos. Isso torna o bcrypt mais exposto a ataques com GPUs e circuitos dedicados do que algoritmos que exigem muita memória. Também é importante usar um custo adequado: custos muito baixos, como 6 a 8, são considerados inseguros hoje.

## 9. Comparação com scrypt e Argon2id

O scrypt, de 2009, e o Argon2id, de 2015 (vencedor da competição Password Hashing Competition), também são funções lentas para senhas, mas exigem quantidades configuráveis de memória, o que dificulta ataques com GPU e ASIC. No scrypt o custo é configurado por  $N$ ,  $r$  e  $p$ ; no Argon2id, por tempo, memória e número de threads. A OWASP aceita o bcrypt com custo 10 ou mais, o scrypt com  $N$  igual a 2 elevado a 17 e o Argon2id com 19 MiB de memória e 2 iterações, e considera o Argon2id a primeira escolha para sistemas novos. O bcrypt continua amplamente usado e suportado em todas as linguagens, em uso desde 1999.

## 10. A implementação da dupla em Python puro

O programa `bcrypt_seminario.py` implementa o bcrypt do zero, sem nenhum import, ou seja, sem `os`, `hashlib`, `hmac`, `time` ou qualquer biblioteca. Ele tem cinco blocos. O bloco A calcula os dígitos de pi com a fórmula de Machin. O bloco B implementa o Blowfish: a classe `Estado`, a função `F`, a cifra de bloco com 16 rodadas de Feistel e a função `expandir_chave`, que é a operação central. O bloco C implementa o base64 do bcrypt, com as funções de codificar e decodificar. O bloco D gera o sal aleatório com um gerador pseudoaleatório simples chamado `SplitMix64`, semeado com endereços de memória; isso serve apenas para demonstração, pois em produção se usa o gerador criptográfico do sistema operacional. O bloco E junta tudo nas funções `hash_bruto`, `gerar_hash` e `verificar_senha`.

Para medir o esforço sem a biblioteca `time`, o programa conta quantas cifras de bloco foram executadas. A verificação da senha compara os bytes em tempo constante com um acumulador de XOR. A demonstração tem cinco partes: confere os dígitos de pi com os valores oficiais do Blowfish; mostra os seis passos do bcrypt com custo 4; confere três vetores de teste oficiais, em que o hash gerado é idêntico ao da biblioteca oficial do bcrypt; mostra o efeito do sal, o login e um ataque de dicionário que descobre uma senha fraca; e demonstra o limite de 72 bytes. Depois há um menu para gerar hash, verificar senha e atacar uma senha ao vivo. Como o Python puro é lento, a demonstração usa custo 4.

## 11. Resumo em dez pontos

1. bcrypt é uma função de hash feita para senhas, criada em 1999 por Provos e Mazieres, baseada no Blowfish.
2. Hash é de mão única: o login recalcula e compara; nunca descriptografa.
3. Funções rápidas como SHA-256 e MD5 são ruins para senhas porque permitem bilhões de palpites por segundo.
4. O sal tem 16 bytes aleatórios por senha e fica dentro do hash; derrota rainbow tables.
5. O custo  $N$  faz o trabalho se repetir 2 elevado a  $N$  vezes; cada +1 dobra o tempo; recomendado

12 ou mais.

6. O hash tem 60 caracteres: versao, custo, sal de 22 caracteres e hash de 31 caracteres.

7. O Blowfish usa rede de Feistel de 16 rodadas, vetor P de 18 subchaves e 4 S-boxes inicializados com digitos de pi.

8. O algoritmo tem 6 passos; o laço de 2 elevado ao custo repeticoes de ExpandKey e onde esta a lentidao.

9. Limitacoes: senha maxima de 72 bytes, byte nulo proibido, pouca memoria (4 KB) comparado a scrypt e Argon2id.

10. Senha fraca continua fraca: o bcrypt so encarece cada palpite.

## **Referencias**

PROVOS, N.; MAZIERES, D. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999. SCHNEIER, B. Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish), 1993. OWASP Password Storage Cheat Sheet. Auth0: Hashing in Action: Understanding bcrypt.